

Cursor Rules深度指南：让AI编程助手真正"懂你"的工程方法

少林码僧 2026-04-23 36 阅读7分钟

关注

前言：为什么你的Cursor体验不如别人？

同样是用Cursor，为什么有些人用得飞起，有些人却频繁纠错、反复解释？

答案往往藏在一个被大多数人忽视的文件里：`.cursorrules`（新版叫`.cursor/rules`）。

这篇文章将系统讲解Cursor Rules的设计哲学、最佳实践，以及如何用Rules构建一个真正"懂你"的AI编程助手。

一、Cursor Rules是什么？

Cursor Rules是一套给AI编程助手的"系统级指令"——在每次对话开始之前，这些规则就已经被注入到AI的上下文中，让AI在整个项目中都遵循你的偏好和规范。

没有Rules时的痛点：

- 每次都要重复解释"我们用TypeScript，不要用JavaScript"
- AI生成的代码风格前后不一致
- AI不了解项目结构，给出通用而非针对性的建议
- 每次对话都从零开始，无法积累项目知识

有了Rules之后：

- AI自动遵循项目技术栈和代码规范
- 输出风格高度一致

- AI"知道"项目架构，给出更精准的建议

二、Rules文件结构（2026最新规范）

旧格式（仍支持）

▼ bash

 体验AI代码助手

 代码解读

复制代码

```
1 项目根目录/
2  └─ .cursorrules  # 单文件，全局生效
```

新格式（推荐）

▼ csharp

 体验AI代码助手

 代码解读

复制代码

```
1 项目根目录/
2  └─ .cursor/
3     └─ rules/
4         └─ global.mdc      # 全局规则（所有文件生效）
5         └─ frontend.mdc   # 前端规则（.tsx/.vue文件生效）
6         └─ backend.mdc    # 后端规则（.py/.go文件生效）
7         └─ testing.mdc    # 测试规则（*.test.*文件生效）
8         └─ database.mdc   # 数据库规则（SQL文件生效）
```

.mdc 格式支持frontmatter来控制规则作用域：

▼ markdown

 体验AI代码助手

 代码解读

复制代码

```
1  ---
2  description: 前端React组件开发规范
3  globs: ["**/*.tsx", "**/*.jsx", "src/components/**"]
4  alwaysApply: false
5  ---
6
7  # React组件开发规范
8
9  ## 组件结构
10 ...
```

三、高质量Rules模板

3.1 全栈TypeScript项目

▼ markdown



体验AI代码助手

代码解读

复制代码

```
1 ---
2 description: 全栈TypeScript项目全局规范
3 alwaysApply: true
4 ---
5
6 # 项目概述
7
8 这是一个基于Next.js 15 + FastAPI的全栈SaaS应用。
9
10 ## 技术栈
11
12 ### 前端
13 - Framework: Next.js 15 (App Router)
14 - UI库: shadcn/ui + Tailwind CSS v4
15 - 状态管理: Zustand
16 - 数据请求: TanStack Query v5
17 - 表单: React Hook Form + Zod
18 - 类型: TypeScript strict mode
19
20 ### 后端
21 - Framework: FastAPI 0.111+
22 - ORM: SQLAlchemy 2.0 (async)
23 - 数据库: PostgreSQL 16
24 - 缓存: Redis 7
25 - 任务队列: Celery + Redis
26 - 认证: JWT (jose库)
27
28 ## 代码规范
29
30 ### 通用原则
31 - 优先使用函数式编程，避免class（除非框架要求）
32 - 变量/函数命名使用camelCase，类型/接口使用PascalCase
33 - 文件命名：组件用PascalCase.tsx，工具函数用kebab-case.ts
34 - 所有字符串使用单引号（Python双引号）
35 - 缩进：2空格（Python 4空格）
36
37 ### TypeScript规范
38 - 禁止使用`any`，用`unknown`替代
39 - 接口优于类型别名（interface > type）
40 - 所有函数必须有明确的返回类型
41 - 使用`satisfies`操作符验证类型
```

```
42
43 ### 错误处理
44 - 前端使用Error Boundary + TanStack Query的onError
45 - 后端统一使用HTTPException，错误码定义在constants/errors.py
46
47 ## 项目结构
48
49 ### 前端结构 (src/)
```

app/ # Next.js页面（App Router） components/ # UI组件（原子组件放ui/，业务组件放business/） hooks/ # 自定义React Hooks lib/ # 工具函数和配置 stores/ # Zustand状态 types/ # TypeScript类型定义

▼ shell

 体验AI代码助手  代码解读 复制代码

```
1
2 ### 后端结构 (api/)
```

routers/ # FastAPI路由（按模块分文件） services/ # 业务逻辑层 models/ # SQLAlchemy模型 schemas/ # Pydantic模式 crud/ # 数据库操作 core/ # 配置和通用功能

▼ markdown

 体验AI代码助手  代码解读 复制代码

```
1
2 ## 生成代码时的注意事项
3 1. 前端组件总是生成对应的TypeScript类型
4 2. API接口同时生成前端类型和后端Schema
5 3. 数据库操作必须使用async/await
6 4. 所有新功能需要考虑错误状态和加载状态
7 5. 生成测试时使用Vitest（前端）和pytest（后端）
```

3.2 Python AI工程项目

▼ markdown

 体验AI代码助手  代码解读 复制代码

```
1 ---
2 description: Python AI工程项目规范
3 globs: ["**/*.py", "notebooks/**"]
4 alwaysApply: false
5 ---
6
7 # Python AI工程规范
8
9 ## 环境信息
```

```
10 - Python: 3.11+
11 - 包管理: uv (不是pip)
12 - 格式化: ruff (不是black)
13 - 类型检查: mypy strict
14
15 ## 技术栈
16 - LLM框架: LangChain 0.3+ / LangGraph 0.2+
17 - Embedding: text-embedding-3-large (OpenAI)
18 - 向量数据库: Qdrant
19 - HTTP客户端: httpx (异步)
20 - 数据处理: pandas/polars
21 - 配置管理: pydantic-settings
22
23 ## 代码风格
24
25 ### 类型注解
26 # 所有函数必须有完整类型注解
27 def process_documents(
28     docs: list[Document],
29     chunk_size: int = 512,
30     overlap: int = 50,
31 ) -> list[TextChunk]:
32     ...
33
34 ### 异步编程
35 # 优先使用async/await, 避免threading
36 async def retrieve_context(
37     query: str,
38     top_k: int = 5,
39 ) -> list[RetrievalResult]:
40     ...
41
42 ### 配置管理
43 # 使用pydantic-settings, 不要硬编码配置
44 class Settings(BaseSettings):
45     openai_api_key: str
46     qdrant_url: str = "http://localhost:6333"
47
48     model_config = SettingsConfig(env_file=".env")
49
50 ## LangChain特定规范
51 - 优先使用LCEL (|管道操作符) 而非旧式Chain
52 - 所有chain使用ainvoke进行异步调用
53 - 使用structured_output而非手动解析JSON
54 - 回调使用langsmith追踪, 不用print调试
55
56 ## 禁止事项
57 - 禁止直接print调试, 使用loguru
58 - 禁止同步调用LLM API (必须async)
59 - 禁止硬编码API Key
```

```
60 - 禁止使用deprecated的LangChain 0.1接口
61
```

3.3 测试专用Rules

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 ---
2 description: 测试编写规范
3 globs: ["**/*.test.ts", "**/*.spec.ts", "**/test_*.py", "tests/**"]
4 alwaysApply: false
5 ---
6
7 # 测试规范
8
9 ## 前端测试 (Vitest + Testing Library)
10
11 ### 测试文件结构
12 describe("ComponentName", () => {
13   // 按功能分组
14   describe("渲染", () => {
15     it("正常渲染不报错", () => {...})
16     it("传入props正确显示", () => {...})
17   })
18
19   describe("交互", () => {
20     it("点击按钮触发回调", async () => {...})
21   })
22
23   describe("边界情况", () => {
24     it("props为空时显示默认值", () => {...})
25   })
26 })
27
28 ### 断言规范
29 - 使用toBeInTheDocument()而非toBeTruthy()
30 - 优先用getByRole/getByLabelText, 最后才用getByTestId
31 - 异步操作使用waitFor + findBy
32
33 ## 后端测试 (pytest)
34
35 ### 固定格式
36 # 测试函数命名: test_动作_条件_期望结果
37 async def test_create_user_with_valid_data_returns_201():
38     ...
39
40 async def test_create_user_with_duplicate_email_returns_400():
41     ...
```

```
42
43 ### 必须使用fixture
44 @pytest.fixture
45 async def authenticated_client(async_client, test_user):
46     """已登录的测试客户端"""
47     token = create_access_token(test_user.id)
48     async_client.headers["Authorization"] = f"Bearer {token}"
49     return async_client
50
51 ### 测试覆盖率要求
52 - 业务逻辑层 (services/) : 90%以上
53 - API路由层 (routers/) : 80%以上
54 - 工具函数 (utils/) : 95%以上
55
```

四、进阶技巧

4.1 项目文档集成

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 # 项目架构 (AI必读)
2
3 ## 数据流向
4 用户请求 → API Gateway → FastAPI → Service层 →
5 Database/Cache → Service层 → 响应序列化 → 用户
6
7 ## 关键设计决策
8 1. 使用CQRS模式：写操作走主库，读操作走只读副本
9 2. 缓存策略：Redis缓存热点数据，TTL=5分钟
10 3. 异步任务：耗时操作（邮件、报表）走Celery队列
11
12 ## 注意事项（避坑）
13 - 用户表和订单表是分库的，跨库JOIN不可用
14 - 文件上传走OSS，不存本地磁盘
15 - 所有价格字段用Decimal，不用float（精度问题）
```

4.2 动态Rules（基于文件类型）

合理使用 **globs** 可以实现规则的智能激活：

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 ---
2 globs: ["src/api/**", "api/**", "**/*.api.ts"]
3 ---
4
5 # API层规范
6 所有API函数必须：
7 1. 有JSDoc注释说明参数和返回值
8 2. 使用统一的错误处理（apiErrorHandler）
9 3. 请求/响应类型从@/types/api导入
10 4. 不包含业务逻辑（只做数据转换和调用）
11
```

4.3 提示AI如何回答

▼ markdown

 体验AI代码助手  代码解读 复制代码

```
1 ## AI回答规范
2
3 ### 代码生成
4 - 生成完整、可运行的代码，不要省略关键部分
5 - 涉及安全（认证/加密/SQL）必须提醒注意事项
6 - 给出代码后，主动说明关键设计选择
7
8 ### 问题解答
9 - 直接给出最佳方案，不要列举N种可能
10 - 如果有多种方案，明确推荐其中一种
11 - 用中文回答，代码注释也用中文
12
13 ### 代码审查
14 - 发现潜在bug时主动指出，不只是回答被问的问题
15 - 指出性能问题和安全隐患
16 - 如果代码可以更简洁，主动给出重构建议
```

五、Rules管理最佳实践

5.1 保持Rules的时效性

Rules需要随着项目演进而更新，建议：

1. 每个Sprint结束时review一次Rules

2. 踩坑时及时将经验写入Rules ("禁止XX操作，因为...")
3. 升级依赖后更新版本号

5.2 团队共享Rules

将 `.cursor/rules/` 目录提交到Git，让团队共享统一的AI配置：

```
bash
1 # .gitignore中不要忽略rules目录
2 # 明确添加到追踪
3 git add .cursor/rules/
4 git commit -m "feat: 添加Cursor Rules配置"
```

5.3 Rules长度控制

Rules不是越长越好——过长的Rules会占用过多上下文窗口，反而降低AI的有效响应质量。

经验值：

- 全局Rules：500-1000字符
- 模块级Rules：300-500字符
- 单个Rules文件不超过2000字符

5.4 常用Rules片段库

```
markdown
1 # 常用片段: React组件模板
2 生成React组件时使用以下模板：
3
4 interface XxxProps {
5   // props定义
6 }
7
8 export function Xxx({ }: XxxProps) {
9   return (
10     <div>
11     </div>
12   )
}
```

```
13 }
14
```

六、完整示例：AI编程项目的Rules配置

▼ bash

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 .cursor/rules/
2 |— global.mdc      # 项目概述、技术栈、通用规范
3 |— python-ai.mdc   # Python/AI开发规范 (*.py)
4 |— docker.mdc      # 容器配置规范 (Dockerfile, docker-compose)
5 |— git-commit.mdc  # 提交信息规范
6 |— security.mdc    # 安全相关 (密钥、认证、注入防护)
```

合理配置Cursor Rules，能让AI编程助手从"懂点代码的机器"变成"真正了解你项目的搭档"。这是2026年做AI辅助编程不可忽视的生产力乘数。

本文规范基于Cursor 0.45+ 版本，Rules格式持续演进，请参考官方文档获取最新信息

标签：

人工智能

LLM

Python

评论 0

登录 / 注册

即可发布评论!



暂无评论数据

目录

收起 ^

前言：为什么你的Cursor体验不如别人？

一、Cursor Rules是什么？

二、Rules文件结构（2026最新规范）

旧格式（仍支持）

新格式（推荐）

三、高质量Rules模板

3.1 全栈TypeScript项目

3.2 Python AI工程项目

3.3 测试专用Rules

四、进阶技巧

4.1 项目文档集成

4.2 动态Rules（基于文件类型）

4.3 提示AI如何回答

五、Rules管理最佳实践

5.1 保持Rules的时效性

5.2 团队共享Rules

5.3 Rules长度控制

5.4 常用Rules片段库

六、完整示例：AI编程项目的Rules配置

精选内容

从"玩具"到"生产力": AI Agent 架构设计的 5 个关键决策

米小虾 · 58阅读 · 1点赞

开源我的 GPT-Image2 生图 Skill，附大量玩法指南

ConardLi · 370阅读 · 8点赞

用AI的方式思考：思维链模式的提示词优化

洞窝技术 · 162阅读 · 1点赞

给 LLM 应用加语义缓存——用 Redis + Embedding 干掉 40% 的重复调用

92year · 51阅读 · 0点赞

🍏 Claude.md 是啥？让你的 AI 助手乖乖听你的话

小白菜的编程日记 · 78阅读 · 1点赞

为你推荐

使用 Cursor 与 AI 协作编程

不二先生不二 | 1年前 | 👁 1.4k | 👍 3 | 💬 评论

人工智能 前端 后端

【cursor】AI时代，打不过就加入

酱79557 | 1年前 | 👁 1.4k | 👍 点赞 | 💬 评论

前端

Windsurf：超越 Cursor 的下一代 AI 编辑器

奇舞精选 | 1年前 | 👁 22k | 👍 65 | 💬 11

前端 Visual ... WindSurf

cursor 使用经验分享，满满干货

幸运蜗牛 | 1年前 | 👁 9.0k | 👍 33 | 💬 9

前端 人工智能 Cursor

"不可错过的Python学习资源: 一起探索AI的未来"

1号程序员 | 2年前 | 👁 223 | 👍 点赞 | 💬 评论

Python

小白到老司机的进化之路：安利一个媲美 Cursor 的AI 编程助手 Aider？

老码小张 | 1年前 | 👁 1.3k | 👍 9 | 💬 评论

前端 后端

github copilot / cursor / windsurf 你该使用哪一个？ AI 编辑器横评

李章鱼 | 1年前 | 👁 15k | 👍 62 | 💬 9

GitHub 后端 WindSurf

Windsurf：超越 Cursor 的下一代 AI 编程助手

加加笔记 | 1年前 | 👁 2.1k | 👍 1 | 💬 1

人工智能

面向前端人的提示词工程指南

LV技术派 | 1年前 | 👁 9.3k | 👍 109 | 💬 10

前端 AIGC

Cursor AI：编程世界的新星，如何迅速崛起？

TodoHacker | 1年前 | 1.2k | 3 | 评论 | Visual ...

掌握 Cursor AI 编辑器：15个核心功能详解

kate人不错 | 1年前 | 5.9k | 10 | 1 | 人工智能 | 响应式... | Cursor

cursor 让程序员体验当甲方爸爸的快乐

陈佬昔编程人生 | 1年前 | 5.9k | 45 | 10 | 人工智能 | Visual ... | AIGC

倚天屠龙：Github Copilot vs Cursor

西滨技术随想 | 2年前 | 4.1k | 8 | 3 | 编程语言 | 通义灵码 | Github ...

如何使用Cursor提高开发效率

smallbone | 1年前 | 15k | 37 | 13 | 人工智能 | GPT | Visual ...

企业级 AI Coding 已经来临， 目前其发展可能已经超越想象，对此我的一些思考

晴小篆 | 1年前 | 1.6k | 6 | 1 | OpenAI | 前端 | 后端

